databricks

Lakeflow Jobs Core Components

LECTURE

Building Blocks of Lakeflow Jobs

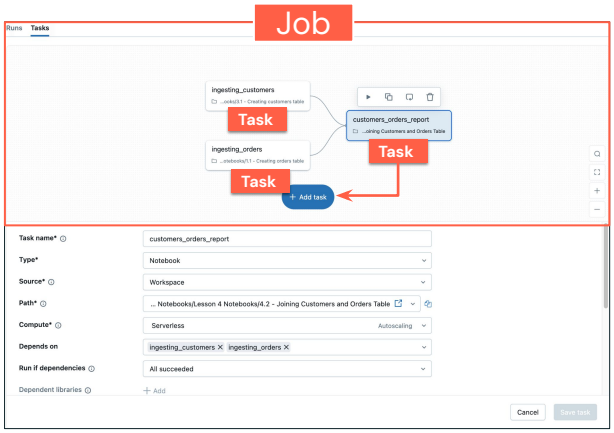
© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's start with the fundamental building blocks that make up every lakeflow job.

Building Blocks of Lakeflow Jobs

Job and Tasks

- A **Job** is the primary resource for:
  - Scheduling
  - Coordinating
  - Running operations such as data processing, ETL, analytics, and machine learning workloads within the Databricks environment.
- A **task** is a **single unit of work** within a job that executes a specific workload such as a notebook, script, query and more.
- Each **job** consists of **one or more tasks**, which are the individual units of work that make up the job



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

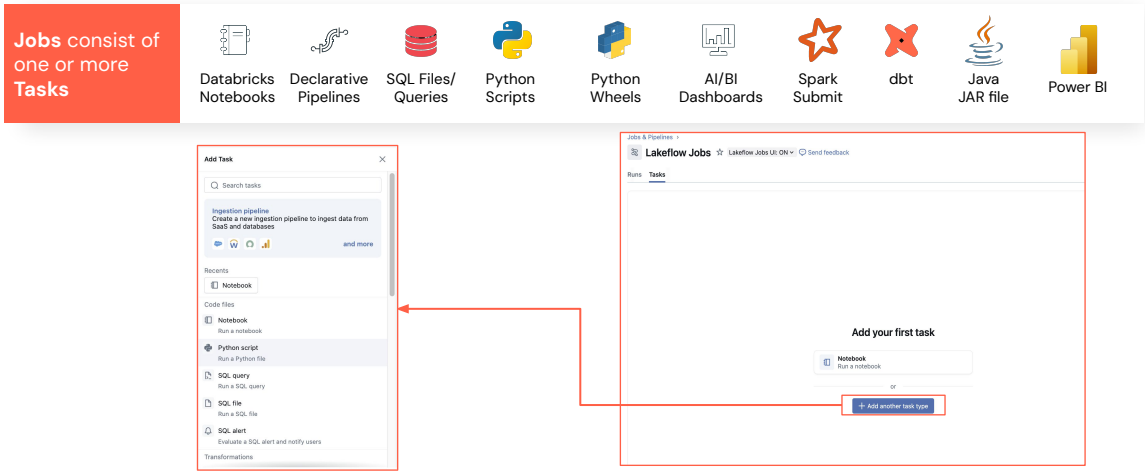
Here are the two most fundamental concepts you need to understand:

A Job is the primary resource for scheduling, coordinating, and running operations such as data processing, ETL, analytics, and machine learning workloads within the Databricks environment. Think of a job as the container that holds your entire workflow.

A task is a single unit of work within a job that executes a specific workload such as a notebook, script, query, and more. Tasks are the individual building blocks that do the actual work.

The relationship is hierarchical: each job consists of one or more tasks, which are the individual units of work that make up the job. The visual shows this clearly - one job containing multiple tasks.

Building Blocks of Lakeflow Jobs



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

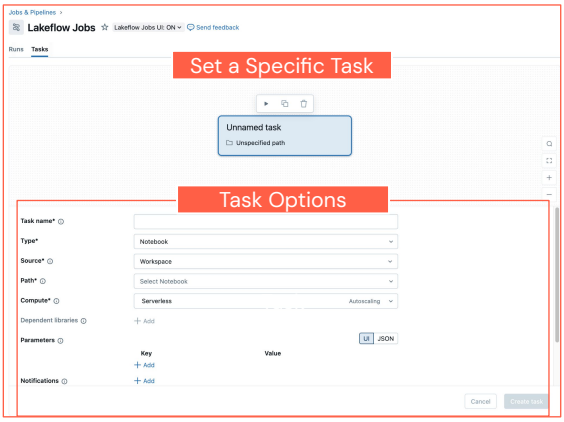
Jobs consist of one or more tasks, and there are many different task types available. You can use Databricks Notebooks in any supported language, Python Scripts, Python Wheels for packaged code, SQL Files and Queries for data transformations, DLT (Declarative Pipelines), dbt for data transformation, Java JAR files, Spark Submit jobs for legacy Spark applications, AI/BI Dashboards for visualization, and even Power BI integration.

This variety ensures that you can orchestrate virtually any type of workload within your job.

Building Blocks of Lakeflow Jobs

Tasks Options

- With **Jobs**, we can set a **particular task**
- We have different **options available** based on the task:
  - Defining path
  - Adding libraries
  - Adding parameters
  - Enabling notification
  - All options are specific to the specific task



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

When you create a job, you can set specific configurations for each particular task. The options available depend on the task type you select. Common configuration options include defining the path to your code, adding libraries, setting parameters, enabling notifications, and configuring retry policies.

These configurations allow you to better orchestrate each particular task according to your specific requirements.

Building Blocks of Lakeflow Jobs

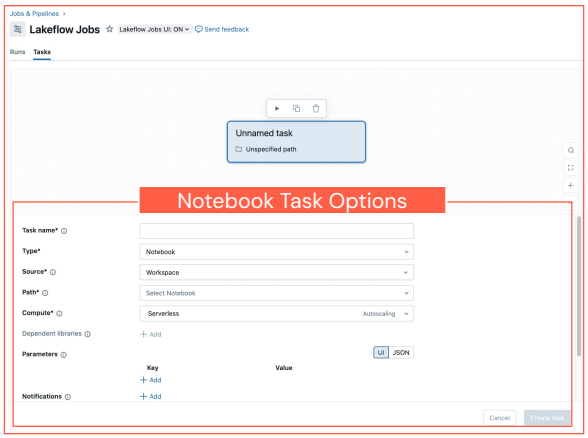
Tasks – Notebook

Based on **task type selected,** you get specific options.

Example

For **Notebook** Tasks:

- Source
- Path
- Compute options
- And More



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Here's a specific example for Notebook tasks. When you select a Notebook task type, you get options like specifying the Source path to your notebook, choosing Compute options (cluster configuration), and many more settings specific to running notebooks.

The interface adapts based on your task type selection, providing relevant configuration options.

Building Blocks of Lakeflow Jobs

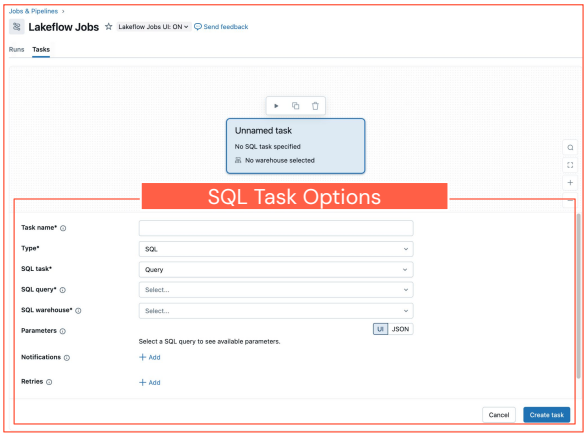
Tasks – SQL

Based on **task type selected,** you get specific options

Example

For **SQL** Tasks:

- Task name
- SQL query
- SQL warehouse



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Similarly, for SQL tasks, you get different options. You can specify the SQL task details, write or reference your SQL query, and select the SQL warehouse that will execute your query.

Each task type provides the specific configuration options needed for that type of workload.

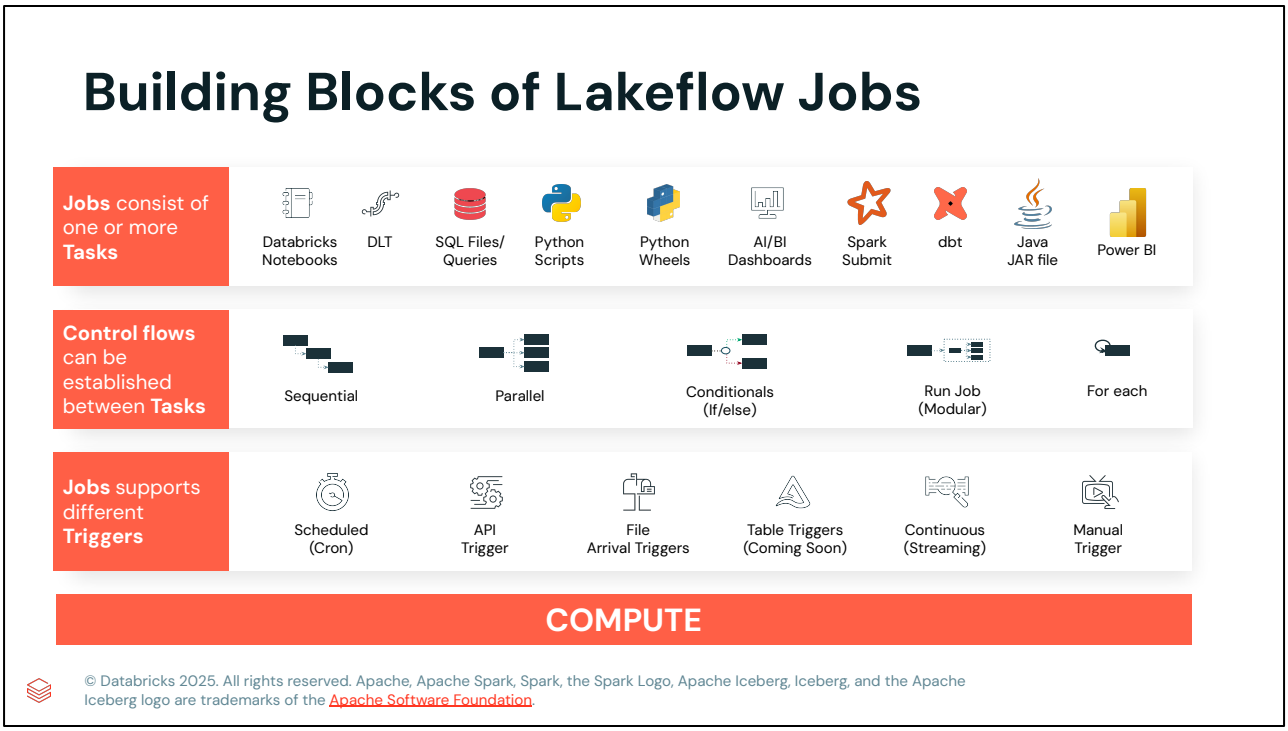
Building Blocks of Lakeflow Jobs

Languages Supported

- Python
- SQL
- Scala
- R
- Java [via a JAR file]

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Lakeflow Jobs supports multiple programming languages: Python, SQL, Scala, R, and Java through JAR files. This broad language support ensures that teams can use their preferred languages and existing code assets.



We already learned about Jobs and Task,

This comprehensive view shows how tasks can be connected with different control flow patterns. You can implement Sequential execution (one after another), Parallel execution (multiple tasks at once), Conditional execution with If/else logic, Run Job tasks for modular design, and For each loops for iterative processing.

Additionally, Jobs supports different trigger types: Manual triggers for on-demand execution, Scheduled triggers using cron expressions, API triggers for programmatic execution, File Arrival triggers for event-driven processing, Table triggers for data change events, and Continuous triggers for streaming workloads.

The compute layer supports all these different orchestration patterns.

Building Blocks of Lakeflow Jobs

Jobs can be executed on different compute

Development, Ad-Hoc Analysis & Exploration	Production Grade & Operational Use-Cases	Simple, Fast, Reliable, Cost Efficient!	Compute for SQL Queries and BI, Serverless by default
Interactive Clusters	**Job Clusters**	**Serverless**	**SQL Warehouse**
Interactive or all-purpose clusters can be shared by multiple users. It is best for performing **ad-hoc analysis, data exploration, or development.** Interactive **should not be used in Production** as they are not cost-efficient.	Job clusters are approximately **50% cheaper** as they terminate when the job ends, reducing resource usage and costs. However, Job clusters are subject to **cloud providers start-up times.** With Databricks Jobs, you can **re-use the same cluster across Tasks for better price performance!**	**Serverless Workflows** are a **fully managed service** that are operationally simpler and more reliable They provide you with **faster clusters & auto-scaling capabilities** providing you with a **better user experience for a lower cost.** With **out of the box performance optimizations**, Serverless provides you with a **lower overall TCO.**	**Purpose-built for SQL queries, dashboards, and BI;** Can be attached to notebook as well. **High concurrency + autoscaling** via Intelligent Workload Management for consistent low latency. **auto-start/auto-stop** and **adjustable cluster size** and max clusters for peak loads **helps control costs**.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Jobs can be executed on different types of compute, and choosing the right compute is crucial for both performance and cost:

Interactive Clusters can be shared by multiple users and are best for ad-hoc analysis, data exploration, or development. However, they should not be used in production as they are not cost-efficient.

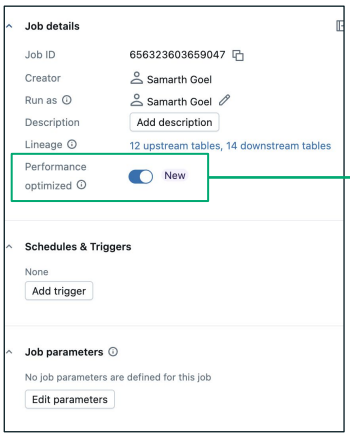
Job Clusters are approximately 50% cheaper as they terminate when the job ends, reducing resource usage and costs. They're ideal for production workloads, though they are subject to cloud provider start-up times. With Databricks Jobs, you can reuse the same cluster across tasks for better price performance.

Serverless provides a fully managed service that is operationally simpler and more reliable. It offers faster clusters and auto-scaling capabilities, providing better user experience for lower cost. With out-of-the-box performance optimizations, Serverless provides lower overall TCO.

SQL Warehouse is purpose-built for SQL queries, dashboards, and BI, and is serverless by default. It offers high concurrency and autoscaling via Intelligent Workload Management, with auto-start/auto-stop and adjustable cluster sizing to help control costs.

Building Blocks of Lakeflow Jobs

Serverless Performance Mode



Use the **Performance optimized** setting in the job details page to choose between **lower cost** and **faster execution** for serverless tasks.

Performance Optimized (Off)

- Standard mode focuses on **cost-efficiency**
- **Longer startup** time (typically 4–6 minutes)
- Best for **non-urgent workloads** with flexible timing

Performance Optimized (On)

- **Enables faster** job startup and execution
- Ideal for **time-sensitive** workloads
- **Applies** only to tasks with **serverless** compute in job

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

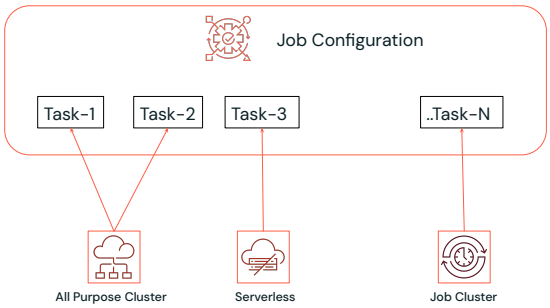
For serverless tasks, you can use the Performance optimized setting to choose between lower cost and faster execution.

Standard Mode focuses on cost-efficiency with longer startup time (typically 4–6 minutes), making it best for non-urgent workloads with flexible timing.

Optimized Mode enables faster job startup and execution, making it ideal for time-sensitive workloads. This setting applies only to tasks with serverless compute in your job.

Building Blocks of Lakeflow Jobs

Selecting Compute



- A Job can have **one or more Tasks** inside it
- Each Task can be assigned its **own compute resource**
- Tasks in the same Job can either:
  - **share** the same compute or
  - use **different** compute as required

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

This diagram illustrates an important principle: A job can have one or more tasks inside it, and each task can be assigned its own compute resource. Tasks in the same job can either share the same compute or use different compute as required.

You might have Task-1 running on an All Purpose Cluster, Task-2 on Serverless, Task-3 on a Job Cluster, and so on. This flexibility allows you to optimize both performance and cost for each specific task.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.